

Person - Car Interaction

This document specifies **every** step, state, condition, and formula in the person-car interaction system.

- Companion overview: README.md
- Gate cheat-sheet: ../STATES_AND_GATES.md

Table of Contents

1. Notation
2. End-to-end pipeline
3. Geometry primitives
4. Mask (segmentation) primitives
5. Motion & continuity primitives
6. Camera ego-motion
7. SAM3 tracking & seg cache
8. Interaction states & gate conditions
9. Segmentation major-axis gate
10. Provisional emission
11. Configuration & fps resolution
12. Outputs

1. Notation

A **box** is axis-aligned $b = (x_1, y_1, x_2, y_2)$ (optionally with a confidence as a 5th entry). A **track** is a map $\{f \mapsto b_f\}$ from frame index to box. Person tracks are indexed by p , car tracks by c .

Symbol	Definition
$\text{ctr}(b)$	Box centre $(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$.
$\text{diag}(b)$	Box diagonal $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} + \varepsilon$, $\varepsilon = 10^{-6}$.
$\text{area}(b)$	$(x_2 - x_1)(y_2 - y_1)$.
W	Start/end window length in frames (window).
start win	First W frames of a track, and end win = last W frames.
M_f	Camera affine transform mapping frame $f-1 \rightarrow f$ (2×3).
Θ	Config thresholds (see §11).

All distances used inside the interaction gates are **normalised by the car’s box diagonal**, so they are scale-invariant to how large/near the car appears.

2. End-to-end pipeline

Per clip, `src/runner.py run(...)`:

1. **Resolve fps.** Read the clip fps and convert all seconds-based thresholds to frame counts via `Config.resolve_for_fps` (§11).
 2. **Track (cache-first).** `Detector.track_video` loads `track_cache/<clip>_sam3seg_s<>.json` if present, else runs SAM3 forward propagation once (person prompt, then car prompt) and writes the cache (§7). Yields **person_tracks**, **car_tracks**, **total_frames**, plus per-frame **mask ellipses** and **silhouette polygons**.
 3. **Snapshot masks, then hide them.** Mask descriptors are cached in an “oracle” and re-revealed **one frame at a time**, so the causal loop never peeks ahead.
 4. **Frame loop** (for $f = 0, 1, \dots$):
 1. Reveal this frame’s person/car boxes and mask descriptors.
 2. **Camera motion:** $M_f = \text{CamMotion.step}$ from frames $f-1, f$ only (§6).
 3. **Driver:** `ProvisionalDriver.push_frame(f, persons, cars, M_f)` ingests the frame and returns any events emitted **this** frame (§10).
 4. Draw overlays. Optionally write the annotated mp4 frame.
 5. **Flush.** Finalize the tail tracks, sort events by (frame, person_id), write `event.json` and `diagnostics.json`.
-

3. Geometry primitives

Source: `src/geometry.py`.

3.1 Overlap (person-over-car)

Intersection area of p and c :

$$I(p, c) = \max(0, \min(x_2^p, x_2^c) - \max(x_1^p, x_1^c)) \cdot \max(0, \min(y_2^p, y_2^c) - \max(y_1^p, y_1^c))$$

$$\text{overlap}(p, c) = \frac{I(p, c)}{\max(\varepsilon, \text{area}(p))}$$

This is intersection over the **person** area (saturates to 1 when the person sits fully inside the car).

3.2 Centre distance in car diagonals

$$d(p, c) = \frac{\|\text{ctr}(p) - \text{ctr}(c)\|_2}{\text{diag}(c)}$$

3.3 Closeness $\in [0, 1]$ — the core proximity score

With `near_dist` the distance (in car diagonals) at which the distance term reaches 0:

$$\text{dist_score}(p, c) = \max\left(0, 1 - \frac{d(p, c)}{\text{near_dist}}\right)$$

$$\text{closeness}(p, c) = \max(\text{overlap}(p, c), \text{dist_score}(p, c))$$

3.4 Distance-IoU (DIoU) $\in [-1, 1]$

Uses intersection over **union** (unlike closeness), so a small person inside a large car scores low and drops further as they move away:

$$\text{IoU} = \frac{I(p, c)}{\text{area}(p) + \text{area}(c) - I(p, c) + \varepsilon}$$

$$\rho^2 = \|\text{ctr}(p) - \text{ctr}(c)\|_2^2, \quad c^2 = (\text{diagonal of the smallest box enclosing both})^2 + \varepsilon$$

$$\text{DIoU}(p, c) = \text{IoU} - \frac{\rho^2}{c^2}$$

4. Mask (segmentation) primitives

Source: `src/geometry.py`.

4.1 Equivalent ellipse of a mask

From binary mask moments m (area = m_{00} , centroid $(\bar{x}, \bar{y})$) and the normalised central moments $\mu_{20}, \mu_{02}, \mu_{11}$:

$$\text{common} = \sqrt{(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2}$$

$$\lambda_1 = \frac{1}{2}(\mu_{20} + \mu_{02} + \text{common}), \quad \lambda_2 = \frac{1}{2}(\mu_{20} + \mu_{02} - \text{common})$$

$$a = 2\sqrt{\lambda_1}, \quad b = 2\sqrt{\lambda_2}, \quad \theta = \frac{1}{2} \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$$

The descriptor is $[\bar{x}, \bar{y}, \text{area}, a, b, \theta]$ (a, b = semi-major / semi-minor lengths, θ = major-axis orientation).

4.2 Offsets in the ellipse frame

For a point (p_x, p_y) with $\Delta = (p_x - \bar{x}, p_y - \bar{y})$:

$$u = \Delta_x \cos \theta + \Delta_y \sin \theta, \quad v = -\Delta_x \sin \theta + \Delta_y \cos \theta$$

4.3 On-major-axis test

With $a' = \max(a, b, 1)$, $b' = \max(b, 1)$ and config reaches `seg_major_reach`, `seg_minor_band`:

$$\text{on_axis} = (|u| \leq \text{seg_major_reach} \cdot a') \wedge (|v| \leq \text{seg_minor_band} \cdot b')$$

i.e. the point lies inside the car's elongated major-axis band (its doors/sides).

5. Motion & continuity primitives

Source: `src/detector.py`.

5.1 Window aggregates

Over a frame set F (missing frames skipped in the numerator, but $|F|$ is the denominator for closeness/overlap):

$$\overline{\text{clos}}(F) = \frac{1}{|F|} \sum_{f \in F} \text{closeness}(p_f, c_f), \quad \overline{\text{ov}}(F) = \frac{1}{|F|} \sum_{f \in F} \text{overlap}(p_f, c_f)$$

$$\overline{\text{DIOU}}(F) = \text{mean}_{f \in F} \text{DIOU}(p_f, c_f), \quad \overline{\text{diag}}_c(F) = \text{mean}_{f \in F} \text{diag}(c_f)$$

Best car for a person over window F : $\text{best_car}(p, F) = \arg \max_c \overline{\text{clos}}(F)$ and returns that max closeness as $c_{\text{start}} / c_{\text{end}}$.

5.2 Radial motion (net approach)

Over the common frames $[f_0, \dots, f_n]$ of p and c :

$$\text{motion}(p, c) = d(p_{f_0}, c_{f_0}) - d(p_{f_n}, c_{f_n})$$

$\text{motion} > 0$ approached the car (enter). Departure magnitude is $-\text{motion}$ (exit).

5.3 Car speed (ground-relative, camera-compensated)

For consecutive frames $f, f+1$ present in the car track, predict where the car centre *would* be under pure camera motion and measure the residual:

$$\widehat{\text{ctr}}_f = M_{f+1} \cdot \text{ctr}(c_f) \quad (\text{or } \text{ctr}(c_f) \text{ if } M_{f+1} \text{ absent})$$

$$v_{\text{car}} = \frac{1}{n} \sum_{(f, f+1)} \frac{\|\text{ctr}(c_{f+1}) - \widehat{\text{ctr}}_f\|_2}{\text{diag}(c_{f_0})}$$

Units: car diagonals per frame. A truly parked car has $v_{\text{car}} \approx 0$ even while the camera moves.

5.4 Person drift & straightness (loiter vs. walk-through)

Over near-frame centres $g_i = \text{ctr}(p_{f_i})$ with mean car diagonal $\bar{D} = \overline{\text{diag}_c}$:

$$\text{net} = \frac{\|g_n - g_0\|_2}{\bar{D}}, \quad \text{path} = \frac{1}{\bar{D}} \sum_i \|g_{i+1} - g_i\|_2$$

$$\text{straightness} = \frac{\text{net}}{\text{path}} \in [0, 1]$$

A **walk-through** has large **net** *and* high **straightness**. A loiterer stays put (small net) or wanders (low straightness).

6. Camera ego-motion

Source: `src/camera.py`. `step(gray_f, boxes_f)`:

1. Build a dynamic-object mask on frame $f-1$: each previous box is padded by `ego_box_pad` and zeroed out (so features come from the background).
2. `goodFeaturesToTrack` on the masked $f-1$ frame (`ego_max_corners`, `ego_quality`, `ego_min_dist`).

3. `calcOpticalFlowPyrLK` tracks those points into frame f . Keep status-1 pairs.
4. If $\geq \text{ego_min_pts}$ survive, fit a partial-affine (similarity) transform with RANSAC:

$$M_f = \text{estimateAffinePartial2D}(\text{prev}, \text{next}) \in \mathbb{R}^{2 \times 3}$$

5. Return M_f (or `None` treated as identity). Uses only frames $f-1, f$.
 M_f feeds `car_speed` (§5.3) so real car motion is separated from panning.

7. SAM3 tracking & seg cache

Source: `src/detector.py track_video, _track_sam3_seg`.

- One SAM3 **session** per clip. Two forward `propagate_in_video` passes: prompt "person" then "car" (`reset_session` between them).
- Per frame, each returned instance with probability $p \geq \text{sam3_score_th}$ becomes a pixel-space box $[xW, yH, (x+w)W, (y+h)H, p]$. Its binary mask is reduced to an ellipse descriptor (§4.1) and simplified silhouette polygons.
- Forward propagation is causal. The cache stores boxes + descriptors so reruns need no GPU and reproduce identical tracks.
- Cache key: `track_cache/<stem>__sam3seg_s<sam3_score_th>.json`.

8. Interaction states & gate conditions

Each person track is judged **online** by `ProvisionalDriver` (see §10): loiter, exit, and enter are each emitted at their earliest causal frame. A state is reported only when **all** its gates pass. Every gate's pass/fail + detail is written to `diagnostics.json`. Let $f_{\text{first}}, f_{\text{last}}$ be the track's first/last frames, N its length, and $\mathcal{D}$ the frame diagonal.

Prerequisite (all states): $N \geq \text{min_track_len}$ (else the track is skipped entirely).

8.1 Exit — *person emerges from the car and leaves*

Let $c^* = \text{best_car}(p, \text{start win})$ and:

$$c_{\text{start}} = \overline{\text{clos}}(\text{start}), \quad c_{\text{end}} = \overline{\text{clos}}(\text{end}), \quad \text{motion} = \text{motion}(p, c^*)$$

$$\text{car_frac} = \frac{\overline{\text{diag}}_c(\text{start})}{\mathcal{D}}, \quad \text{overlap_th} = \begin{cases} \text{on_car_overlap_th_big} & \text{car_frac} \geq \text{big_car_frame_ratio} \\ \text{on_car_overlap_th} & \text{otherwise} \end{cases}$$

$$\text{emerged} = (\overline{\text{ov}}(\text{start}) > \text{overlap_th}) \vee (c_{\text{start}} \geq \text{on_car_near_th})$$

Gate	Condition	Motivation
<code>not_at_clip_edge</code>	$f_{\text{first}} > \text{edge_margin}$	Ignore tracks that exist from frame 0 (no observable arrival).
<code>at_car_at_start</code>	$c_{\text{start}} > \text{near_th}$	Person is genuinely at the car when first seen.
<code>emerged_from_car</code>	<code>emerged</code>	Person physically overlapped the car (came out of it), relaxed for big cars.
<code>closeness_drop</code>	$(c_{\text{start}} - c_{\text{end}}) > \text{delta} \vee (\overline{\text{DIoU}}(\text{start}) - \overline{\text{DIoU}}(\text{end})) > \text{exit_diou_drop}$	Overall proximity decreased.
<code>car_stationary</code>	$v_{\text{car}} \leq \text{car_max_speed}$	The car was parked, not driving off.

8.2 Enter — *person reaches the car and gets in*

Only evaluated when the track was **not** an exit and a nearby car exists. Let $c^* = \text{best_car}(p, \text{end win})$ and:

$$c_{\text{end}} = \overline{\text{clos}}(\text{end}), \quad c_{\text{start}} = \overline{\text{clos}}(\text{start}), \quad \text{motion} = \text{motion}(p, c^*)$$

$$\text{reached} = (\overline{\text{ov}}(\text{end}) > \text{overlap_th}) \vee (c_{\text{end}} \geq \text{on_car_near_th})$$

Gate	Condition	Motivation
<code>vanishes_mid_clip</code>	$f_{\text{last}} < \text{total_frames} - 1 - \text{min_absence}$	Person disappears <i>into</i> the car mid-clip, not at the clip's end.
<code>at_car_at_end</code>	$c_{\text{end}} > \text{near_th}$	At the car when last seen.
<code>reached_car</code>	<code>reached</code>	Physically reached the car body (big-car relaxation as in exit).
<code>closeness_rise</code>	$(c_{\text{end}} - c_{\text{start}}) > \text{delta}$	Overall proximity increased — a genuine approach.

Gate	Condition	Motivation
<code>moved_closer</code>	<code>motion > move_min</code>	Real net radial approach, not incidental drift.
<code>car_stationary</code>	$v_{\text{car}} \leq \text{car_max_speed}$	Car parked while they approached.

Emitted at frame f_{last} . Enter is the only state with irreducible latency (§10).

8.3 Loiter — *person lingers near a parked car*

Candidate car $c^* = \arg \max_c |\{f : \text{closeness}(p_f, c_f) > \text{loiter_near_th}\}|$. Let `near_count` be that count over near-frames F^* , and $(\text{net}, \text{straightness}) = \text{person_drift}(F^*)$.

`walk_through` = $(\text{net} > \text{loiter_max_net_drift}) \wedge (\text{straightness} > \text{loiter_straight_th})$

Gate	Condition	Motivation
<code>near_long_enough</code>	<code>near_count</code> $\geq$ <code>loiter_min_frames</code>	Lingered near the car long enough to count.
<code>car_stationary</code>	$v_{\text{car}} \leq \text{car_max_speed}$	Nearness to a <i>parked</i> car.
<code>not_walk_through</code>	$\neg \text{walk_through}$	Reject pass-throughs (long, straight paths).

Emitted at frame F_0^* (first near-frame) with `duration` = `near_count`. Note loiter uses the stricter `loiter_near_th` = 0.60 vs. the enter/exit `near_th` = 0.35.

9. Segmentation major-axis gate

Applied **after** an enter/exit passes the box gates (`ProvisionalDriver` $\rightarrow$ `Detector.axis_gate`). It can only *remove* events, never add them.

How it works: 1. Use the early frames (start window) for exits, late frames (end window) for enters. 2. For each frame with a car mask: check if the person is on the car's long axis (doors/sides). 3. Count how many frames pass: if $\frac{\text{passing frames}}{\text{total frames}} \geq \text{seg_axis_frac}$ (default 0.40), keep the event. 4. If no car mask exists in the window: keep the event by default.

Motivation: box proximity alone can fire on someone *in front of / behind* a large car. Requiring alignment with the car’s long axis keeps events at the doors/sides where real entering/exiting happens.

10. Provisional emission

Source: `src/driver.py` `ProvisionalDriver`. All three states are emitted at their earliest moment.

Finalize delay (latency). A track is *finalized* once the person has been absent long enough that no future frame can change its verdict:

```
finalize_delay = min_absence
```

Finalize runs **no** classification — `_finalize_track` only closes the track and records its diagnostics from the events that already fired.

State	When emitted	How
loiter	The frame the near-count first crosses <code>loiter_min_frames</code>	<code>_check_loiter_exit</code> grows per-(person,car) near-frame lists each frame. <code>_prov_loiter</code> picks the car maximising ($ F $, $\overline{\text{clos}}$), then applies <code>car_stationary</code> and <code>not_walking_past</code> . Reported <code>frame</code> =first near-frame, <code>duration</code> =near-count so far.

State	When emitted	How
exit	The frame a trailing window first shows departure	<code>_exit_base</code> computes the fixed start-window context once (must pass edge, <code>at_car_at_start</code> , <code>emerged</code> , <code>car_stationary</code>). Each later frame checks $(c_{\text{start}} - c_{\text{end}}) > \text{delta} \vee (\text{DIOU drop}) > \text{exit_diou_drop}$ over the current end window, then the axis gate on the start window.
enter	Once the person has been absent for the whole <code>min_absence</code> horizon	While the person is on screen, standing at a car is indistinguishable from having just entered. <code>_prov_enter</code> waits until they stay gone (<code>f_idx - f_last min_absence</code>), then applies the same box gates as <code>exit</code> 's reverse (<code>at_car_at_end</code> , <code>reached</code> , <code>closeness_rise</code> , <code>moved_closer</code> , <code>car_stationary</code>) and the axis gate. Enter and exit are mutually exclusive.

`push_frame` returns the events newly emitted for the current frame. `flush` finalizes any still-open tracks at end of stream and publishes the diagnostics.

11. Configuration & fps resolution

Source: `config/settings.py`. Fields ending in `_sec` are in **seconds**. `Config.resolve_for_fps(f)` converts them to per-clip frame counts (and per-frame speeds) so a threshold means the same real-world duration at any frame rate:

$$\begin{aligned}
\text{window} &= \max(1, \text{round}(\text{window_sec} \cdot f)) & \text{edge_margin} &= \max(0, \text{round}(\text{edge_margin_sec} \cdot f)) \\
\text{min_track_len} &= \max(2, \text{round}(\text{min_track_sec} \cdot f)) & \text{min_absence} &= \max(0, \text{round}(\text{min_absence_sec} \cdot f)) \\
\text{loiter_min_frames} &= \max(1, \text{round}(\text{loiter_min_sec} \cdot f)) & \text{car_max_speed} &= \frac{\text{car_max_speed_per_sec}}{f}
\end{aligned}$$

Threshold reference (defaults):

Field	Default	State(s)	Meaning
min_track_sec	0.20 s	all	Minimum track length to judge.
near_dist	1.3	all (closeness)	Distance (car diagonals) at which closeness $\rightarrow 0$.
car_max_speed_per_sec	0.35 diag/s	all	Max ground-relative car speed (car_stationary).
window_sec	0.17 s	enter, exit	Start/end window span.
near_th	0.35	enter, exit	“At the vehicle” closeness.
delta	0.18	enter, exit	Min closeness change (rise/drop).
on_car_overlap_th	0.15	exit, enter	Min window overlap (emerged/reached).
on_car_overlap_th_big	0.15	exit, enter	Relaxed overlap for big/near cars.
big_car_frame_ratio	0.45	exit, enter	car/frame diagonal ratio counted “big”.
on_car_near_th	0.60	exit, enter	Closeness that alone satisfies emerged/reached.
exit_diou_drop	0.045	exit	Min start $\rightarrow$ end DIOU drop confirming departure.
edge_margin_sec	0.10 s	exit	Ignore exits this soon after clip start.

Field	Default	State(s)	Meaning
<code>min_absence_sec</code>	0.20 s	enter	Person must vanish this long before clip end.
<code>move_min</code>	0.35	enter	Min net radial approach/departure.
<code>loiter_min_sec</code>	2.0 s	loiter	Min time near the car.
<code>loiter_near_th</code>	0.60	loiter	Closeness counted as “near”.
<code>loiter_max_net_drift</code>	0.10	loiter	Max net displacement for a loiter.
<code>loiter_straight_th</code>	0.85	loiter	Straightness above which it’s a walk-through.
<code>seg_major_reach,</code> <code>seg_minor_band,</code> <code>seg_axis_frac</code>	1.35, 1.4, 0.40	enter, exit	Segmentation major-axis gate.
<code>seg_strict</code>	False	enter, exit	Reject events whose window has no mask data.
<code>sam3_score_th</code>	0.5	tracking	Min SAM3 instance probability.

12. Outputs

Per clip under `runs/<clip>/`:

- **event.json** — every emitted signal. Each record is an `Event` (`src/events.py`): `person_id`, `car_id`, `type` (`enter/exit/loiter`), `frame`, `c_start`, `c_end`, `motion`, `duration`, and `metrics` (the condition values — `value/th/pass` — that accepted it).
- **diagnostics.json** — per-track: the `result` (states emitted, or `none`) and the `events` list with their `metrics`. Built directly from the emitted events. no re-classification is run at finalize.
- **<clip>.mp4** — annotated video (unless `--no-save`).